In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.ensemble import IsolationForest
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
```

In [3]:
```
import pandas as pd
url = "https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv"
df = pd.read_csv(url)
df.head()
```

Out[3]:

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278

5 rows × 31 columns

In [4]:
```
df.info()
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

Out[4]:

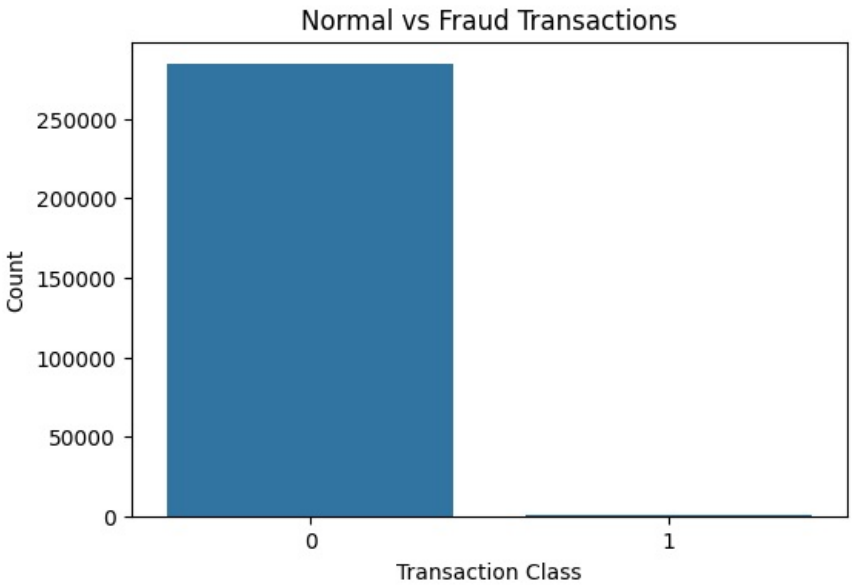
	Time	V1	V2	V3	V4	V5	V6	V7	
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2
mean	94813.859575	1.175161e-15	3.384974e-16	-1.379537e-15	2.094852e-15	1.021879e-15	1.494498e-15	-5.620335e-16	1
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00	1.237094e+00	1
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01	-4.355724e+01	-7
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01	-5.540759e-01	-2
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	4.010308e-02	2
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01	5.704361e-01	3
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	1.205895e+02	2

8 rows × 31 columns

In [5]:
```
df['Class'].value_counts()
```

Out[5]:
```
Class
0    284315
1       492
Name: count, dtype: int64
```

In [6]:
```
plt.figure(figsize=(6,4))
sns.countplot(x='Class', data=df)
plt.title("Normal vs Fraud Transactions")
plt.xlabel("Transaction Class")
plt.ylabel("Count")
plt.show()
```



In [7]:
```
scaler = StandardScaler()
df['Amount_scaled'] = scaler.fit_transform(df[['Amount']])
df = df.drop(['Amount','Time'], axis=1)
```

In [8]:
```
X = df.drop('Class', axis=1)
y = df['Class']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

In [9]:
```
model = IsolationForest(
    n_estimators=100,
    contamination=0.002,
    random_state=42)
model.fit(X_train)
```

Out[9]:
```
▾ IsolationForest ⓘ ⓘ
▸ Parameters
```

In [10]:
```
predictions = model.predict(X_test)
```

In [11]:
```
predictions = np.where(predictions == -1, 1, 0)
```

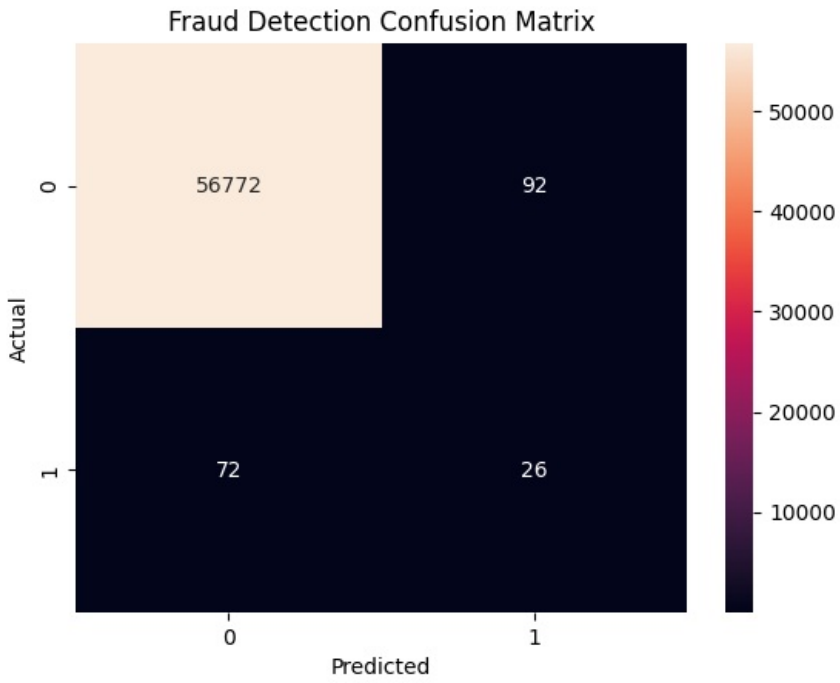
In [12]:
```
print(classification_report(y_test, predictions))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.22      0.27      0.24        98

    accuracy                           1.00     56962
   macro avg       0.61      0.63      0.62     56962
weighted avg       1.00      1.00      1.00     56962
```

In [13]:
```
cm = confusion_matrix(y_test, predictions)
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Fraud Detection Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



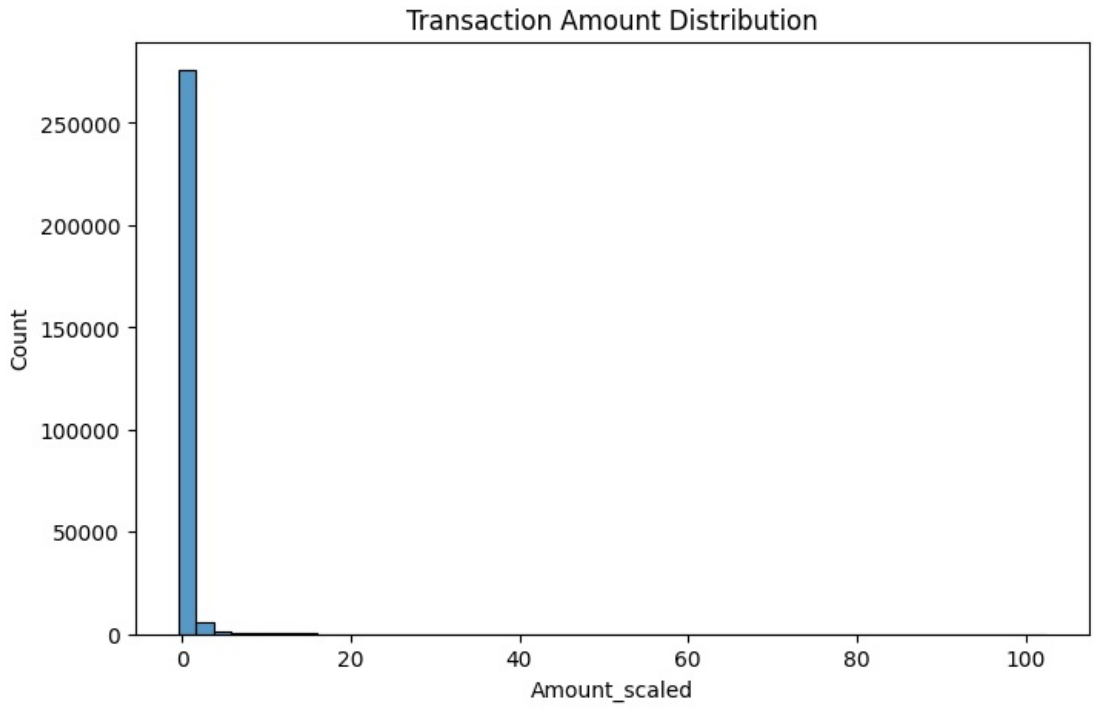
In [14]:
```
fraud_alerts = X_test[predictions == 1]
fraud_alerts.head()
```

Out[14]:

	V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	...	
43428	-16.526507	8.584972	-18.649853	9.505594	-13.793819	-2.832404	-16.701694	7.517344	-8.507059	-14.110184	...	-
219257	-29.942972	-25.831782	-16.227512	6.690679	-20.787846	13.085694	17.256623	-9.161746	5.003041	-2.431466	...	-2
20453	-10.930244	6.000305	-10.746348	1.319919	-4.576451	-1.194886	-2.561206	5.249172	2.739458	3.342491	...	
229036	-32.058119	-48.060856	-6.939376	10.856608	26.647697	-20.869626	-16.407368	2.192752	0.451464	0.946152	...	1
128606	-9.323036	8.154362	-5.743131	1.136134	-4.440984	4.138796	-13.729788	-19.119754	-4.105163	-5.691141	...	

5 rows × 29 columns

In [15]:
```
plt.figure(figsize=(8,5))
sns.histplot(df['Amount_scaled'], bins=50)
plt.title("Transaction Amount Distribution")
plt.show()
```



In [16]:
```
plt.figure(figsize=(8,5))
sns.boxplot(x=y_test, y=X_test['Amount_scaled'])
plt.title("Fraud vs Normal Transaction Amount")
plt.show()
```

In [17]:
```
fraud_rate = predictions.sum() / len(predictions)
print("Detected Fraud Rate:", fraud_rate)
```

```
Detected Fraud Rate: 0.002071556476247323
```

In [ ]: